Improving Long-Range Graph Propagation with MPSSM Based Preconditioning

Mohamad Essa
M.Sc. Computer Science Student
BGU University
esamo@post.bgu.ac.il

Abstract

We propose `MPSSM-PCG`, a learned preconditioner for graph-structured linear systems based on MPSSM [6]. We evaluate the method as a preconditioner for PCG and as a learned long-range propagation operator for shallow GCNs. On graph-derived systems, `MPSSM-PCG` achives convergence results comparable to `NeuralIF`[12] while requiring substantially lower setup time. In addition, we show that the learned operator improves long-range propgation on ECHO-SSSP, allowing shallow GCNs to match or outperform deeper local GCNs. Code is provided in `https://github.com/MohamadCS/MPSSM-PCG`.

1 Introduction

Graph neural networks [11] are a very powerful tool for learning on graphs. They differ from standard MLP in that they mix node representations according to the graph structure, which makes them vaiable for graph based application.

In particular, message-passing neural networks (MPNN) [9] have recived substantial attention over the last decade. MPNN layer updates node representations by aggregating information from neighbor nodes followed by a feature update (such as MLP).

While MPNN are effective for propagating local graph information, global information propagation remain challenging. For information to travel between nodes that are k-hops apart, a local MPNN generally requires at least k propagation layers. However, repeated propagation arrises two main challenges in GNN: oversmoothing and oversquashing [1] .

Oversmoothing [1] happens when a GNN based network stacks many MPNN layers, in which repeated averaging smooths node embeddings, and as a result they become very similar.

Oversquashing [1] When we repeatadly apply MPNN layers to node signals, an exponential amount of information needs to pass through a fixed size embedding vector.

Our goal is to design a shallow MPNN that uses a global propagation matrix for implict long-range propagation, which allows us to achive long-range propgation with fewer MPNN layers. To do that, we use a global propgation matrix like $(I + \alpha L)^{-1}$ (see appendix) to replace the local propgation matrix (often normalized adjacency matrix).

However, the inverse of sparse matrices like $I + \alpha L$ usually dense, which makes it harder construct and apply.

To address this, we learn an efficent operator to invert $M_\theta \approx I + \alpha L$. This allow us to approximate the effect of applying the global propagation operator, without the need of explicitly computing it. This provides both long-range feature propagation and the efficiency of shallow MPNNs.

Preprint.

In this paper, we design a preconditioning model called `MPSSM-PCG` based on MPSSM [6], and use it to precondition the global operator $I + \alpha L$.

1.1 Problem Setup

Preconditioning Given an SPD linear system $Kx = b$, a preconditioner is a matrix $P \approx K^{-1}$ that is efficent to apply. Ideally we require the preconditioned system to satisfy $\kappa(PK) \ll \kappa(K)$, where κ denotes the condition number. In SPD settings, this corresponds to better clustering of the eigenvalues of the preconditioned system, which accelerates the convergence of iterative solvers like PCG.

In our settings, instead of learning P directly we learn an easily invertable approximation $M_\theta(K) \approx K$ and define $P_\theta(K) = M_\theta(K)^{-1} \approx K^{-1}$.

We use the parametrization $M_\theta = L_\theta L_\theta^\top$ where L_θ follows the IC(0) sparsity pattern (Eq.6).

Shallow Long-Range MPNN Given a graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, denote its adjacency matrix by A, where $A_{ij} = A_{ji} = \mathbb{I}[(i,j) \in \mathcal{E}]$, and its normalzied laplacian matrix by $L = I - D^{-1/2}AD^{-1/2}$. Considering the operator $K = I + \alpha L$, our goal is to learn an efficent operator to invert $M_\theta(K) \approx K$ so that using shallow MPNN layer 1 with $P_\theta(K) = M_\theta^{-1}(K)$ as its shift operator

$$Y = \sigma(P_\theta(K)XW) \tag{1}$$

approximates long-range propagation. Here W is a learnable feature mixing matrix, σ is a non-linear activation function and X contains the node signals.

2 Related Work

2.1 Long Range Propagation

Attention Based Method Transformers [19] model the depenencies between feature sequence using self-attention mechansim. Graph nodes can be represented as a seqeunce of tokens, such that the computed attention matrix is treated like a new weighted adjacency matrix. The problem with this approach is that the transformer is not aware of the graph structure. Works like Graphormer [20] use positional and structural embeddings to inject the graph structure. GraphGPS [17] computes global attention to learn depenencies between distant nodes, and uses a local MPNN to learn depenencies between nearby nodes.

Implicit GNN [2, 3] IGNN achive long-range propgation by solving the equalibrium equation $X = f_\theta(X, A)$, where A is the adjacency matrix and X are the node signals. For instance, the original IGNN model [10] finds X such that $X = \sigma(WXA + b)$. They solve it by repeatdly applying the equation using the same learned weight matrices until convergance, allowing information to propagate through long distances without the need for deep propagation.

2.2 Learning Preconditioners

[13] learnes a preconditioner $P = f_\theta(A, b)$ for the discritzed PDE linear system $Ax = b$. They treat A as an adjacency matrix, having A_{ij} as an edge feature and b_i as a node feature. They encode the graph into a hidden represnentation using MLP encoder, then they follow it by multiple MPNN layers and pass into a decoder that predicts a matrix M that is used to construct the preconditioner P. In order to gaurantee symmetry they learn LL^T.

[4] contructs a two-layer sparse linear neural network whose weights are the entries of the incomplete Cholesky factor L with fixed sparisty pattern. The first layer applies $L^\top$ while the second applies L.

[12] learns a lower-triagonal matrix $L_\theta(A)$ with strictly positive diagonal, where the final preconditioner becomes $L_\theta(A)L_\theta^\top(A)$. They do that by firstly propagating information using MPNN layers of the lower-triagonal part of A, then they do the same with the upper triagonal part of A. Finally the edge embeddings become the entries of a sparse matrix triagonal factor.

[7] uses a GCN based method for learning a network M that acts as an approximation to the inverse. They do that by first scaling the input vector b and pass it into an encoder for a hidden represnentation, then they pass it into multiple GCN layers, decode it back to its original dimenstion and rescale it.

Most methods we've looked at directly learn a preconditioner. [18] proposed to use GNNs to learn a correction for the given matrix. Given a lower triagonal IC factor L of A, their goal is to learn $L(\theta) = L + \alpha\ \mathrm{GNN}(\theta, L)$. They use simple encodig, propagation using MPNN and decoding back.

Algebric multigrids (AMG) are also used for learning preconditioners. AMG contructs a hierarchy of coarse represnentations directly from the given matrix. [15] uses GNN based architcture to learn the prolongation weights between the coarse and fine levels. [14] combined algebric multigrid principles with a transformer based architicture to learn a multi-scale preconditioning operator.

[5] learns a fixed pattern sparse approximation inverse $M = GG^\top$. The differnece with from other methods is that they minimize spectral objectives of M rather than Frobenius norm.

3 Method

Given the system $K = I + \alpha L$, we define the sparsity pattern of K as $(A_s)_{ij} = \mathbb{I}[K_{ij} \neq 0]$, $(D_s)_{ii} = \sum_j (A_s)_{ij}$, and the propagation operator

$$S = D_s^{-1/2} A_s D_s^{-1/2}$$

For each matrix row i we construct the feature vector

$$u_i = \left[\frac{K_{ii}}{s}, \frac{\sum_{i \neq j} |K_{ij}|}{s}, \frac{d_i}{\bar{d}}, p_i \right] \tag{2}$$

where $s = \frac{1}{n} \sum_{i=1}^n |K_{ii}|$ and $\bar{d} = \frac{1}{n} \sum_{i=1}^n d_i$ when d_i is the number of nonzero off-diagonal entries in row i and $p_i \in [-1, 1]$ is the normalzied row position.

We combine the feature vectors $\{u_i\}$ to contruct the feature matrix $U = [u_1, \ldots, u_n]^\top \in \mathbb{R}^{n \times 4}$.

MPSSM We use MPSSM [6] as our backbone for learning the matrix L_θ as it gives a natural way of seperating S and U . For an MPSSM block, we initialzie the hidden state as $X_0 = 0$, and update it for T recurrent step according to

$$X_{t+1} = S X_t W + UB \qquad t = 0, \ldots, T-1$$

The same input projection UB is injected at every recurrent step and W is shared across all steps of the block. After the final recurrence we apply

$$Z = \texttt{MLP}(X_T) \tag{3}$$

We stack B MPSSM blocks. The first block operates on U

$$H^{(1)} = \texttt{GraphNorm}(\texttt{MPSSM}_1(S, U)) \tag{4}$$

and

$$H^{(l)} = \texttt{GraphNorm}(H^{(l-1)} + \texttt{MPSSM}^{(l-1)}(S, H^{(l-1)})) \tag{5}$$

This results in $H = H^{(B)}$ which will be used to predict L_θ.

Incomplete Factorization Prediction Similar to [12] we restrict the lower-triagonal part of L_θ (excluding the diagnoal) to the IC(0) sparisty pattern

$$E_0 = \{(i, j) | i > j, K_{ij} \neq 0\} \tag{6}$$

For every $(i, j) \in E_0$ we construct the edge feature

$$e_{ij} = \left[H_i || H_j || \frac{K_{ij}}{s} \right] \tag{7}$$

and predict

$$(L_\theta)_{ij} = \texttt{MLP}_{\text{edge}}(e_{ij}) \tag{8}$$

We predict the diagonal independly. To guarantee the the diagonal is postive, we use the transformation

$$(L_\theta)_{ii} = \exp\left(c \cdot \texttt{MLP}_{\text{diag}}(H_i)\right) \tag{9}$$

where c is a constant[1].

Now since L_θ is a lower triagonal matrix with strictly positive diagonal we get $M_\theta = L_\theta L_\theta^\top$ is a PD matrix and hence invertable and can be used for PCG.

Training Our training method is similar to [14, 12]. Instead of directly minimizing the objective $\|K - L_\theta L_\theta^\top\|_F^2$, which can be quite expensive as the matrix dimesntions increase, we sample k random vectors $x_i \sim \mathcal{N}(0, I)$, construct the matrix $X = [x_1, \ldots, x_k]$ and minimize

$$\mathcal{L} = \frac{\|KX - L_\theta L_\theta^\top X\|_F^2}{\|KX\|_F^2} \tag{10}$$

Preconditioned Graph Propagation We achive long-range propgation by using the global operator $K^{-1} = (I + \alpha L)^{-1}$ as our propgation operator. We firstly approximate K using `MPSSM-PCG` and obtain the incomplete factorization $M_\theta = L_\theta L_\theta^\top \approx K$, then define the propgation opertor as $P_\theta = M_\theta^{-1}$ and plug it into a conventional GCN layer

$$H^{(l+1)} = \sigma(P_\theta H^{(l)} W^{(l)})$$

Refer to appendix (6) for GCN architicture details

4 Evaluation

4.1 Experiment Setup

Our evaluation consists of two parts. First we directly evaluate the model as a preconditioner for solving graph-structured linear equations using preconditioned conjugate gradient (PCG (1)). Second, we use the learned preconditioning operator as a propagation operator of a shallow GCN and evaluate whether it improves long-range propagation on long-range graph learning tasks.

For a graph $\mathcal{G}$ we construct the system $K = I + \alpha L$, where $L = I - D^{-1/2} A D^{-1/2}$ is the normalized laplacian, and $\alpha > 0$ (see A) controls the effective propagation range .

Preconditioning Training Both `MPSSM-PCG` and `NeuralIF` are trained under the same protocol. We train the models using the objective in Eq.10 with 10 Gaussian probes per matrix and evaluate the best validation checkpoint on the test split. At test time, each unseen system is solved 5 times using indepenedly generated RHS. We terminate PCG when $\|r\|_2 / \|b\|_2 < \texttt{tol} = 10^{-6}$ or after 1000 iterations. Refer to the appendix for per expirement hyperparameters.

Long-range Propagation After training `MPSSM-PCG` on a dataset. We freeze `MPSSM-PCG`'s parameters. Then, we train two identical conventional GCN models, that only differ in the propagation matrix. One time we train with $\hat{A}$, and the other with P_θ. For ECHO-SSSP [16], both models are trained using MSE (Eq.15) and selected according to valdiation MAE (Eq.17) at best validation checkpoint. Refer to the appendix for per expirement hyperparameters.

Note that we do each expirement 3 times using the seeds $\{0, 1, 2\}$.

[1]We found that having an indepenent decoder for the diagonal is more numerically stable

4.2 Preconditioner Performence

We first evaluate the quality of the preconditioner through its effect on the convergence of PCG. `MPSSM-PCG` is compared agains `NeuralIF`, Jacobi and the unpreconditioned identity baseline. For each test system, we sample $x \sim \mathcal{N}(0, I),\ b = Kx$ which provides the exact solution for evaluating both convergence and solution accuracy. Instead of directly constructing M_θ^{-1}, we apply the preconditioner in two steps $L_\theta y = r$ then $L_\theta^\top z = y$. Complete residual and solution-error statisctics are provided in the appendix.

Table 1: Preconditioning performence on ECHO-SSSP graph systems using the operator $K = I + 8L$. Mehods are evaluated on their PCG performence on unseen test system. Results are averaged over three seeds $\{0, 1, 2\}$. Time units are in ms.

Method	Conv. % ↑	PCG Iteration ↓	Setup Time ↓	Solve Time ↓	Total Time ↓
Identity	$100\% \pm 0.0$	21.86 ± 0.00	$\mathbf{0.001} \pm 0.000$	2.225 ± 0.011	2.225 ± 0.011
Jacobi	$100\% \pm 0.0$	21.50 ± 0.00	0.008 ± 0.001	2.240 ± 0.010	2.249 ± 0.010
NeuralIF	$100\% \pm 0.0$	$\mathbf{8.73} \pm 0.02$	0.722 ± 0.001	$\mathbf{1.587} \pm 0.007$	2.309 ± 0.005
MPSSM-PCG	$100\% \pm 0.0$	8.93 ± 0.03	0.274 ± 0.007	1.636 ± 0.004	$\mathbf{1.910} \pm 0.004$

The results show that both `NeuralIF` and `MPSSM-PCG` substantially reduce the number of PCG iterations compared with classical precondtioners in PCG iteration count. `NeuralIF` acheives slighty lower iteration count than `MPSSM-PCG` . However `MPSSM-PCG` has approximately 62% lower setup time (C.4) than `NeuralIF`, resulting in approximatly 17% reduction in the total time (C.4). We see very similar results using Peptides graphs [8] (See table 9 in appendix).

4.3 Long-Range Propagation Performence

Next we evaluate whether the learned preconditioner can be used as an effective long-range propagation operator in a shallow graph neural network.

Table 2: Effect of the propgation parameter α on ECHO-SSSP. The table shows the test MAE for and PGCN per α value. PGCN uses 1 layer for propgation.

α	PGCN MAE ↓
1.0	0.1255 ± 0.000571
2.0	0.1174 ± 0.000126
4.0	0.1106 ± 0.000609
8.0	0.1070 ± 0.000488

In table (2) we perform simple abliation to evalute the effect of the propagation parameter α. As established by (A), increasing α increases the effective propgation range of $(I + \alpha L)^{-1}$. Therefore we expect that larger values of α to be benificial on long-range benchmarks like ECHO-SSSP [16]. The results shown in 2 is consistent with this interpretation, as the test MAE decreases as α increases. We can also notice diminishing returns past $\alpha = 4$.

Table 3: Depth comparison on ECHO-SSSP. The table shows the test MAE for local GCN and PGCN at different propagation depths. PGCN uses a learned preconditioning operator with $\alpha = 8.0$.

Layers	Local GCN MAE ↓	PGCN MAE ↓
1	0.1421 ± 0.000030	0.1070 ± 0.000488
2	0.1321 ± 0.000091	0.0846 ± 0.000891
4	0.1112 ± 0.000242	0.0791 ± 0.001215
8	0.0748 ± 0.000437	0.0684 ± 0.001254

In the second expirement we evaluate the effect of replacing the local propagation operator $\hat{A}$ with the learned operator P_θ. As shown in 3, PGCN achives substantially better performence with less layers. A one layer of PGCN achives an MAE of 0.107 while outperforming four layers of local GCN that achived MAE of 0.111. As depth increases the local GCN substantially improves and closes the gap, however PGCN still outperforms it at every depth.

5 Conclusion

We introduced `MPSSM-PCG`, a learned preconditioner for graph-structured linear systems based on MPSSM. `MPSSM-PCG` achives preconditioning quality comparable to an existing learned preconditioner while requireing substantially lower setup time. Furthermore, we showed that the learned operator $P_\theta \approx (I + \alpha L)^{-1}$ can be used as a long-range graph propagation operator. The learned operator substantially improves the performence of shallow GCNs enabling long-range propagation to be achieved using fewer layers.

5.1 Limitations

Main limitation arrise from our limited computation budget. First, the multilayer long-range propgation in table 3 is conducted using a single value of α, so evalutaing the relation between propogation depth and α more extensively could provide a better image of the effect of the learned operator.

Second, we considered operators of the form $I + \alpha L$ only. Other long-range graph operators like e^{-tL}, $L^\dagger$ could also be considered, as they could be more suitable for differnet graph-learning tasks.

Finally `MPSSM-PCG` is specifically designed around graph-structured sparse systems and was evaluated on systems computed from graph-learning datasets. Its effectiveness on more general sparse SPD matrices remains to be investigated.

References

[1] Adrian Arnaiz-Rodriguez and Federico Errica. Oversmoothing, Oversquashing, Heterophily, Long-Range, and more: Demystifying Common Beliefs in Graph Machine Learning, February 2026.

[2] Justin Baker, Qingsong Wang, and Cory Hauck. Implicit Graph Neural Networks: A Monotone Operator Viewpoint.

[3] Justin M. Baker, Qingsong Wang, Martin Berzins, Thomas Strohmer, and Bao Wang. Monotone Operator Theory-Inspired Message Passing for Learning Long-Range Interaction on Graphs. In *Proceedings of The 27th International Conference on Artificial Intelligence and Statistics*, pages 2233–2241. PMLR, April 2024.

[4] Joshua Dennis Booth, Hongyang Sun, and Trevor Garnett. Neural Acceleration of Incomplete Cholesky Preconditioners.

[5] Francesco Brarda, Tianshi Xu, Vassilis Kalantzis, Rui Peng Li, and Yuanzhe Xi. Factored Sparse Approximate Inverse Preconditioning via Spectral Optimization, June 2026.

[6] Andrea Ceni, Alessio Gravina, Claudio Gallicchio, Davide Bacciu, Carola-Bibiane Schonlieb, and Moshe Eliasof. Message-Passing State-Space Models: Improving Graph Learning with Modern Sequence Modeling. https://arxiv.org/abs/2505.18728v2, May 2025.

[7] Jie Chen. Graph Neural Preconditioners for Iterative Solutions of Sparse Linear Systems, January 2025.

[8] Vijay Prakash Dwivedi, Ladislav Rampášek, Mikhail Galkin, Ali Parviz, Guy Wolf, Anh Tuan Luu, and Dominique Beaini. Long Range Graph Benchmark, November 2023.

[9] Justin Gilmer, Samuel S. Schoenholz, Patrick F. Riley, Oriol Vinyals, and George E. Dahl. Neural Message Passing for Quantum Chemistry, June 2017.

[10] Fangda Gu, Heng Chang, Wenwu Zhu, Somayeh Sojoudi, and Laurent El Ghaoui. Implicit Graph Neural Networks, June 2021.

[11] William L Hamilton. Graph Representation Learning.

[12] Paul Häusner, Ozan Öktem, and Jens Sjölund. Neural incomplete factorization: Learning preconditioners for the conjugate gradient method, October 2024.

[13] Yichen Li, Peter Yichen Chen, Tao Du, and Wojciech Matusik. Learning Preconditioner for Conjugate Gradient PDE Solvers, September 2023.

[14] Zhihao Li, Di Xiao, Zhilu Lai, and Wei Wang. Neural Preconditioning Operator for Efficient PDE Solves. https://arxiv.org/abs/2502.01337v2, February 2025.

[15] Ilay Luz, Meirav Galun, Haggai Maron, Ronen Basri, and Irad Yavneh. Learning Algebraic Multigrid Using Graph Neural Networks. In *Proceedings of the 37th International Conference on Machine Learning*, pages 6489–6499. PMLR, November 2020.

[16] Luca Miglior, Matteo Tolloso, Alessio Gravina, and Davide Bacciu. Can You Hear Me Now? A Benchmark for Long-Range Graph Propagation, February 2026.

[17] Ladislav Rampášek, Mikhail Galkin, Vijay Prakash Dwivedi, Anh Tuan Luu, Guy Wolf, and Dominique Beaini. Recipe for a General, Powerful, Scalable Graph Transformer, January 2023.

[18] Vladislav Trifonov, Alexander Rudikov, Oleg Iliev, Yuri M. Laevsky, Ivan Oseledets, and Ekaterina Muravleva. Learning from Linear Algebra: A Graph Neural Network Approach to Preconditioner Design for Conjugate Gradient Solvers, February 2025.

[19] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention Is All You Need, August 2023.

[20] Chengxuan Ying, Tianle Cai, Shengjie Luo, Shuxin Zheng, Guolin Ke, Di He, Yanming Shen, and Tie-Yan Liu. Do Transformers Really Perform Bad for Graph Representation?, November 2021.

A Interpretation of $(I + \alpha L)^{-1}$

Throughout the paper we have used $K^{-1} = (I + \alpha L)^{-1}$ as our long-range propgation operator. We now show mathematically that K^{-1} indeed does multi-hop propagation of the normalized adjecacny matrix $\hat{A}$.

Notice that $K = I + \alpha L = (1 + \alpha)I - \alpha\hat{A}$ and so using Neumann expansion

$$K^{-1} = \frac{1}{1+\alpha}\left(I - \frac{\alpha}{1+\alpha}\hat{A}\right)^{-1} = \frac{1}{1+\alpha}\sum_{k=0}^{\infty}\left(\frac{\alpha}{1+\alpha}\right)^{k}\hat{A}^{k} \tag{11}$$

The usage of Neumann expansion is valid since for $\alpha > 0$

$$\rho\left(\frac{\alpha}{1+\alpha}\hat{A}\right) = \frac{\alpha}{1+\alpha}\rho\left(\hat{A}\right) \leq \frac{\alpha}{1+\alpha} < 1 \tag{12}$$

A conventional GCN layer propgates using $\hat{A}$

$$H^{(l+1)} = \sigma(\hat{A}H^{(l)}W^{(l)})$$

This only does 1-hop propagation.

Replacing $\hat{A}$ with K^{-1} is equivilant to

$$H^{(l+1)} = \sigma(K^{-1}H^{(l)}W^{(l)}) = \sigma\left(\frac{1}{1+\alpha}\sum_{k=0}^{\infty}\left(\frac{\alpha}{1+\alpha}\right)^{k}\hat{A}^{k}H^{(l)}W^{(l)}\right)$$

Define

$$w_k = \frac{1}{1+\alpha}\left(\frac{\alpha}{1+\alpha}\right)^{k} \tag{13}$$

Notice that larger α corresponds to slower weight decay. Therefore we can achive longer effictive propagation by increasing α.

B Additional Method Details

B.1 PCG Algorithm

Algorithm 1 Preconditioned Conjugate Gradient

Require: SPD Matrix A, RHS b, preconditioner P, initial guess x_0, tolerance ϵ, maximum iterations K

$x \leftarrow x_0$
$r \leftarrow b - Ax$
$z \leftarrow P(r)$
$p \leftarrow z$
for $k = 0, \ldots, K-1$ **do**
  $q \leftarrow Ap$
  $r' \leftarrow \langle r, z \rangle$
  $\alpha \leftarrow \frac{r'}{\langle p, q \rangle}$
  $x \leftarrow x + \alpha p$
  $r \leftarrow r - \alpha q$
  if $\frac{\|r\|_2}{\|b\|_2} < \epsilon$ **then**
    return x
  end if
  $z \leftarrow P(r)$
  $\beta \leftarrow \frac{\langle r, z \rangle}{r'}$
  $p \leftarrow z + \beta p$
end for
return x

B.2 `MPSSM-PCG` implementation

Algorithm 2 MPSSM Block

Require: Propgation matrix S, Matrix Features U, SSM steps T

$X_0 \leftarrow \mathbf{0}$
for $s = 0, \ldots T-1$ **do**
  $X_{s+1} \leftarrow SX_sW + UB$
end for
return $\texttt{MLP}(X_T)$

Algorithm 3 MPSSM

Require: Propgation matrix S, Matrix Features U, SSM steps T, SSM Blocks B, Dropout p

$H_0 \leftarrow \texttt{GraphNorm}(\texttt{MPSSM}_0(U, S))$
for $b = 0, \ldots, B-1$ **do**
  $H_{b+1} \leftarrow \texttt{GraphNorm}(H_b + \texttt{Dropout}_p(\texttt{MPSSM}_b(H_b, S)))$
end for
return H_B

Algorithm 4 MPSSM-PCG

Require: System K

$S \leftarrow D^{-1/2} A_S D^{-1/2}$ ▷ A_s is the sparisty pattern (Eq.6) of K

$U \leftarrow \texttt{matrix\_node\_features}(K)$ ▷ $U = [u_1, \ldots, u_n]^\top$ when u_i is defined as in Eq.2

$H \leftarrow \texttt{MPSSM}(S, U)$

$E_0 \leftarrow \{(i,j) | i > j, K_{ij} \neq 0\}$

$L_\theta \leftarrow \mathbf{0}$

for all $(i,j) \in E_0$ **do**

  $e_{ij} \leftarrow [H_i || H_j || K_{ij}/s]$

  $(L_\theta)_{ij} = \texttt{MLP}_{\text{edge}}(e_{ij})$

end for

for $i = 1, \ldots, n$ **do**

  $\delta_i \leftarrow \texttt{MLP}_{\text{diag}}(H_i)$

  $(L_\theta)_{ii} \leftarrow e^{c\delta_i}$ ▷ In our implementation $c = 0.1$

end for

return L_θ

Algorithm 5 Apply MPSSM-PCG Preconditioner

Require: Sparse Factor L, Residual r

$y \leftarrow \texttt{solve\_triangular}(L, r, \text{upper} = F)$ ▷ Pytorch's `torch.linalg.solve_triangular`

$z \leftarrow \texttt{solve\_triangular}(L^\top, y, \text{upper} = T)$

return z

B.3 GCN Architicture

Algorithm 6 GCN For ECHO-SSSP

Require: Normalized adjecacny $\hat{A}$, Node Embeddings X, Layers L, Dropout p

$H_0 \leftarrow \texttt{MLP}_{\text{encoder}}(X)$

for $i = 0, \ldots, L-1$ **do**

  $R_t \leftarrow H_t$

  $H_t \leftarrow \texttt{ReLU}(\texttt{LayerNorm}(\hat{A} H_t W_t))$

  $H_t \leftarrow \texttt{Dropout}_p(H_t) + R_t$

end for

return $\texttt{MLP}_{\text{out}}(H_L)$

Algorithm 7 PGCN For ECHO-SSSP

Require: Sparse Factor L_θ, Node Embeddings X, Layers L, Dropout p

$H_0 \leftarrow \texttt{MLP}_{\text{encoder}}(X)$

for $i = 0, \ldots, L-1$ **do**

  $R_t \leftarrow H_t$

  $H_t \leftarrow \texttt{ReLU}(\texttt{LayerNorm}(\texttt{apply}(L_\theta, H_t) W_t))$ ▷ When `apply` as in alg.5

  $H_t \leftarrow \texttt{Dropout}_p(H_t) + R_t$

end for

return $\texttt{MLP}_{\text{out}}(H_L)$

C Evaluation

C.1 Hardware Configuration

We run all the tests on Ryzen 9600X CPU, 32GB of RAM and an AMD Radeon 9070 XT GPU with 16 GB of VRAM.

C.2 Software and Libraries

Specific libraries are provided using ‘requirements.txt‘.

C.3 Expirements Hyperparameters

Table 4: Hyperparameters used in MPSSM-PCG ECHO-SSSP/Peptides-struct preconditioning expirement

Hyperparameter	Value
Optimizer	Adam
Learning Rate	10^{-3}
Train Epochs	50
Checkpoint Selection	Best Validation loss
Hidden Dimesntion	16
Blocks	2
Steps	4
Normalization	`GraphNorm`
Dropout	0

Table 5: NeuralIF Hyperparameters ECHO-SSSP/Peptides-struct preconditioning expirement

Hyperparameter	Value
Optimizer	Adam
Learning Rate	10^{-3}
Train Epochs	50
Checkpoint Selection	Best Validation loss
message_passing_steps	4
global_features Dimesntion	0
latent_size	8
skip_connections	True
activation	ReLU
aggregate	None
decode_nodes	False
normalize_diag	False
graph_norm	True
two_hop	False
edge_features	1

This configuration is the closest configuation to what the authors of NeuralIF [12] chose. We just used 4 message passing steps instead of 3 to have a close parameter count to ours.

Table 6: Hyperparameters used in MPSSM-PCG ECHO-SSSP expirement

Hyperparameter	Value
Optimizer	Adam
Learning Rate	10^{-4}
Train Epochs	30
Checkpoint Selection	Best Validation loss
Hidden Dimesntion	16
Blocks	2
Steps	4
Normalization	`GraphNorm`
Dropout	0

Table 7: GCN hyperparameters for ECHO-SSSP long-range propagation expirements

Hyperparameter	Value
Optimizer	Adam
Learning Rate	10^{-5}
Train Epochs	300
Checkpoint Selection	Best Validation loss
Hidden Dimenstion	384
Normalization	`LayerNorm`
Dropout	0.1

GCN layers used in the expirements are stated in each table.

The total parameter count of `MPSSM-PCG` is 2594 while `NeuralIF` is 2488.

C.4 Metrices

Setup time Measures the cost of contructing the preconditioner for a unseen system K. For `MPSSM-PCG`, this consists of constructing the matrix representation, running the MPSSM encoder and decoding L_θ. Simliarly, for `NeuralIF` this is the forward pass for constructing L_θ. Preconditiong training is not part of the setup time.

Since the tested preconditioners only rely on K, for each graph we sample $N_{rhs} = 5$ random RHS and reuse the computed preconditioner to solve the 5 systems, so the actual setup time presented in tables is

$$t_{setup} = \frac{t_{\text{preconditioner construction}}}{N_{rhs}} \tag{14}$$

Solve Time Measures the execution time of PCG 1 and the applications of the preconditioner. MPSSM-PCG and NeuralIF preconditioners are applied using 5.

Total Time The sum of the setup time and solve time.

Mean Squared Error For a graph G with predictions $\hat{\mathbf{y}}$ and targets $\mathbf{y}$ containing N targets

$$\text{MSE}(G) = \frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2 \tag{15}$$

for a mini batch B

$$\text{MSE}(B) = \frac{1}{|B|}\sum_{G\in B}\text{MSE}(G) \tag{16}$$

Mean Absolute Error

$$\text{MAE} = \frac{\sum_G \sum_{v\in V_G} |y_v - \hat{y}_v|}{\sum_G |V_G|} \tag{17}$$

During training, the training loss gives equal weights to each graph (regardelss of the number of nodes). In evaluation MAE give equal weight to each target node.

C.5 Additional Expirements

In addition to Tabel 1 we include both the relative residual

$$\frac{\|b - K\hat{x}\|_2}{\|b\|_2} \tag{18}$$

and the relative solution error

$$\frac{\|x - \hat{x}\|_2}{\|x\|_2} \tag{19}$$

Table 8: Preconditioning performence on ECHO-SSSP graph systems. Mehods are evaluated on their PCG performence on unseen test system. Results are averaged over three seeds $\{0, 1, 2\}$.

Method	Rel. Error ↓	Rel. Reisdual ↓
Identity	$2.094 \cdot 10^{-6} \pm 2.305 \cdot 10^{-9}$	$7.410 \cdot 10^{-7} \pm 1.065 \cdot 10^{-9}$
Jacobi	$2.112 \cdot 10^{-6} \pm 7.335 \cdot 10^{-9}$	$7.348 \cdot 10^{-7} \pm 1.985 \cdot 10^{-9}$
NeuralIF	$1.286 \cdot 10^{-6} \pm 6.873 \cdot 10^{-9}$	$4.725 \cdot 10^{-7} \pm 1.534 \cdot 10^{-9}$
MPSSM-PCG (ours)	$1.289 \cdot 10^{-6} \pm 1.076 \cdot 10^{-8}$	$4.725 \cdot 10^{-7} \pm 2.207 \cdot 10^{-9}$

Table 9: Preconditioning performence on peptides-struct graph systems $K = I + 8L$, with 2000/500/500 split. Mehods are evaluated on their PCG performence on unseen test system. Results are averaged over three seeds $\{0, 1, 2\}$. Timing are presented in ms.

Method	Conv. % ↑	PCG Iteration ↓	Setup Time (ms) ↓	Solve Time (ms)↓	Total Time (ms) ↓
Identity	100.0 ± 0.0	21.90 ± 0.00	0.001 ± 0.000	2.244 ± 0.002	2.245 ± 0.002
Jacobi	100.0 ± 0.0	21.64 ± 0.01	0.009 ± 0.002	2.266 ± 0.004	2.275 ± 0.006
NeuralIF	100.0 ± 0.0	9.00 ± 0.01	0.732 ± 0.005	1.947 ± 0.064	2.679 ± 0.062
MPSSMPCG	100.0 ± 0.0	9.23 ± 0.02	0.299 ± 0.004	2.015 ± 0.053	2.314 ± 0.052

Table 10: Preconditioning performence on peptides-struct graph systems with 2000/500/500 split. Mehods are evaluated on their PCG performence on unseen test system. Results are averaged over three seeds $\{0, 1, 2\}$.

Method	Rel. Error ↓	Rel. Reisdual ↓
Identity	$2.220 \cdot 10^{-6} \pm 5.722 \cdot 10^{-9}$	$7.815 \cdot 10^{-7} \pm 1.628 \cdot 10^{-9}$
Jacobi	$2.031 \cdot 10^{-6} \pm 3.599 \cdot 10^{-9}$	$6.932 \cdot 10^{-7} \pm 1.051 \cdot 10^{-9}$
NeuralIF	$1.679 \cdot 10^{-6} \pm 8.283 \cdot 10^{-9}$	$6.393 \cdot 10^{-7} \pm 4.363 \cdot 10^{-9}$
MPSSMPCG	$1.602 \cdot 10^{-6} \pm 2.746 \cdot 10^{-8}$	$5.961 \cdot 10^{-7} \pm 6.647 \cdot 10^{-9}$